Object Deep Dive | size, call by value vs Reference | Shallow vs Deep Copy | JAVA full cource. # 14.

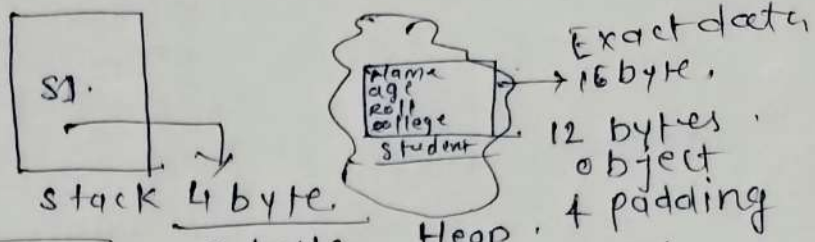
Deep dive object →

```
class Student {
String name; → 4 byte
int age; → 4 byte.
int rollno; → 4 byte
String colleg; → 4 byte.
```

Total. 16 byte

Student s1 = new Student();

in Memory



Total 32 bytes.

Object size →

1. Header size → object Header
2. Exact fields
3. Padding

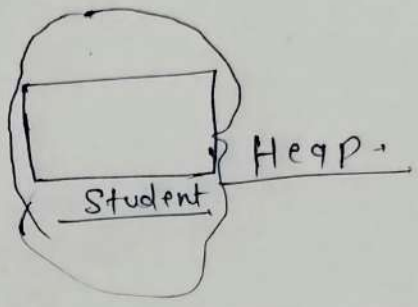
1. Object Headers.
   ↓
   Meta data.
   - Mark words
   - class Pointer

   > 4 bytes or 8 bytes.

   4+8 = 12 byte

   - Lock
   - Synchronization
   - Garbage Collection.
   etc.

   8 byte.



2. Exact data. →

```
class student {
String name; → 4 bytes
int age; → 4 bytes.
int rollno; → 4 bytes.
String college; → 4 bytes.
}
```

Exact data 16 bytes.

3. Padding → it is a optional.
   ↳ 8 bytes multiple

CPU → = object Header + Exact data.

CPU works as 8 byte multiple

12 + 16

= 28 bytes

Add 4 bytes. → padding

Total 32 bytes.

```
class person {
byte ag;
}
```

⇒ object Metadata, 12 bytes.
+ Exact data. = 1 bytes.
= 13 bytes.

Adding. 3 padding

Total = 16 bytes.

Because of the. CPU works as the. 8 byte multiple

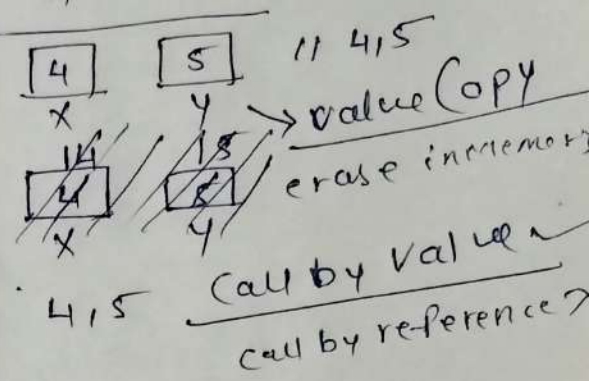
student s1;
↓
userdefine data Type
↓
data Type

Call by value vs Call by Reference

// Call by value → There is.

```
Public class Day 14 {
  Public static void main (string [] args) {
  int x = 4;
  int y = 5;
  System.out.Print-ln (x + " , " + y);
  addTen (x, y);
  System.out.Print-ln (x + " , " + y);
  }
  static void add Ten (int x, int y) {
    x = x + 10;
    y = y + 10;
  }
}
```

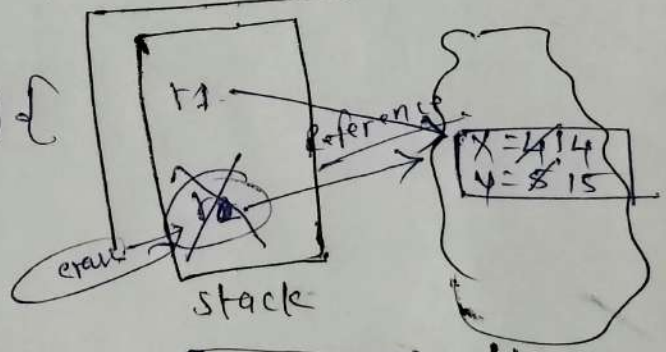
• 4, 5
• 4, 5



// call by Reference → There is no call by reference in java.

```
Public class Day 14 {
  Public static void main (string [] args) {
  Random r1 = new Random (4, 5);
  System.out.Print-ln (r1.x + " , " + r1.y);
  addTen (r1);
  System.out.Print-ln (r1.x + " , " r1.y);
  }
  static void add Ten (Random r) {
  r.x = r.x + 10;
  r.y = r.y + 10;
}}
```

output
4, 5
14, 15

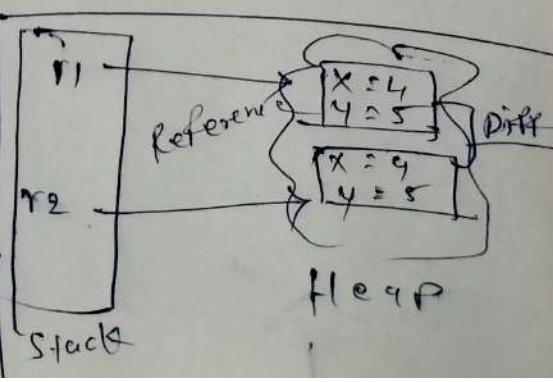



```
class Random {
  int x;
  int y;
  → Random (int x, int y) {
      this.x = x;
      this.y = y; }}
}
```

Deepcopy new.
~~Random s2 = Random (r1)~~
```
Random (Random r) {
  this.x = r.x;
  this.y = r.y;
}}
```

• JAVA
  → Call by value ?
  → Call by reference ? X

[C/C++] Both. but JAVA only for call by value




```
Random r1 = new Random (4, 5);
Random r2 = new Random (r1);
Random r3 = r1 → shallow copy
```
// Deepcopy